Runi: Locality-Guided GPU Reoptimization for Batched Dynamic Capacitated Vehicle Routing

Nithin Hanumanthu (CS23B025)
Research Project Guide: Prof. Rupesh Nasre
Department of Computer Science and Engineering, Indian Institute of Technology Madras

Abstract. Reoptimization of vehicle routes after customer cancellations and new requests offers an opportunity to reuse high-quality existing tours rather than solve the full capacitated vehicle routing problem again. We present *Runi*, a CUDA-based batch repair method combining centroid-guided candidate caching, exclusive route-worker warps, direct ready-route dispatch, multi-block scheduler scans, and spatially sparse local-search refinement. The optimizer separates fast feasible repair from quality recovery through tail exchanges, relocations, SWAP*, and intra-route 2-opt. In recorded experiments, Runi completes valid balanced batches through 12,800 events on a 500,000-customer instance. A refinement sweep identifies 15 passes as a practical quality–latency compromise for the tested workloads. We report comparisons with Senay Patel's dynamic VND implementation [2], static seqMDS [1], and full-instance best-known CVRP references while distinguishing unmatched optimization budgets from direct dynamic-update evidence.

1. Introduction

A vehicle-routing plan is rarely the last plan that an operator needs. After routes have been prepared, some customers may cancel, while new customers require service. Solving the entire problem again can improve solution quality, but it also discards the computational value of the existing routes. At the other extreme, inserting every new customer into the first feasible route gives rapid updates at the expense of travel distance. The useful middle ground is to *repair an incumbent solution quickly and then selectively recover its quality.*

This paper presents **Runi**, a CUDA-based method for batched reoptimization of the capacitated vehicle routing problem (CVRP). Its design follows the sequence in which a changed plan must be repaired. First, the obsolete customers are removed. The remaining routes are candidates for the new requests, but assessing every route repeatedly would be wasteful. Runi therefore uses spatial locality to shortlist likely routes and caches these decisions. Independently scheduled warps then perform actual route modifications, while a final, sparse local-search stage restores solution quality. The key engineering issue is not merely launching more GPU threads: the algorithm must ensure that the threads perform useful work without being stalled by queue polling, redundant scans, or cross-block synchronization.

We make three contributions. First, Runi defines a batched delete–add–refine pipeline with route-exclusive GPU repairs and spatially cached candidate selection. Second, it couples multi-block centroid evaluation with an adaptive scratch-memory policy, keeping scheduler parallelism independent from the maximum reserved route length. Third, it studies both update-scale behavior and the quality–latency trade-off in the number of refinement passes. Rather than treating static CVRP best-known solutions (BKS) as dynamic-event results, we distinguish final-state solution quality from the cost of performing an update.

2. Problem Statement

Consider a single depot 0 and a set of active customers V_t at update epoch t. Customer i has demand $q_i > 0$ and Euclidean location $\mathbf{p}_i = (x_i, y_i)$, and all vehicles have capacity Q. A solution $\mathcal{R}_t$ is a set of ordered routes $r = (0, v_1, \ldots, v_{L_r}, 0)$ such that every active customer appears exactly once, no inactive customer is served, and

$$\sum_{i \in r} q_i \le Q \qquad \forall r \in \mathcal{R}_t. \tag{1}$$

We minimize the sum of travel distances,

$$F(\mathcal{R}_t) = \sum_{r \in \mathcal{R}_t} \sum_{(u,v) \in E(r)} d(u,v),$$
$$d(u,v) = \operatorname{round}\big(\|\mathbf{p}_u - \mathbf{p}_v\|_2\big). \tag{2}$$

with rounding applied separately to each edge.

An update consists of a deletion set D_t and an addition set A_t, taking V_t to $V_{t+1} = (V_t \setminus D_t) \cup A_t$. The initial routes are given. Runi processes the entire transaction in the order *all deletions, all additions, then refinement*; this semantics does not depend on the order in which records were listed in the input. Unmodified portions of routes may be changed during refinement, so this is an *unrestricted planning-snapshot* model.

The benchmark is an unrestricted reoptimization snapshot: completed travel, vehicle motion, and committed route prefixes are not modeled, and the objective is evaluated solely on the final active customer set.

3. From Static seqMDS to Dynamic Updates

3.1 Why not maintain the minimum spanning tree?

The starting point for this investigation was the static *seqMDS* algorithm [1]. Its construction first obtains a minimum spanning tree (MST), explores randomized depth-first traversals of that tree, splits the resulting visit order into capacity-feasible routes, and repeatedly searches for improvements. This is effective for a new, static instance, but it is not automatically a good strategy when only a small fraction of customers change.

An initial direction was to maintain the MST incrementally as customers were inserted or deleted. The attraction is clear: if the tree could be repaired cheaply, perhaps the updated routes could be constructed without restarting the static algorithm. Profiling shifted our focus. In the examined static execution, approximately **99% of time** was spent in the repeated optimization *superloop*, rather than the one-time MST construction. A faster MST update would therefore accelerate only a small portion of execution. The more promising intervention was to avoid repeating the whole route-construction and improvement process after each batch.

This observation led to an architectural change rather than a more elaborate dynamic-tree data structure: retain the current solution, identify a small set of promising modifications, and invoke the expensive improvement neighborhoods selectively. The next sections explain how Runi turns that principle into a GPU pipeline.

4. Runi: Locality-Guided GPU Repair

4.1 From cancellations to unplaced requests

Deletions are uncomplicated in comparison with additions. Runi marks all cancelled customers and compacts every affected route. Surviving customer order is retained at this stage, and route load, location metadata, and centroid information are refreshed.

New customers form a device-resident *frontier* of requests that still need a route. During *ejection*, a worker makes room for a newly inserted customer by removing an existing customer from a route; that displaced customer returns to the frontier to be assigned elsewhere. This common representation is important: instead of implementing a separate repair procedure for each possible failure, every request returns to the same decision process until placed or declared infeasible. A scheduler decides *which route should be tried*; a worker then decides *how the route should be changed*. These roles are deliberately separate.

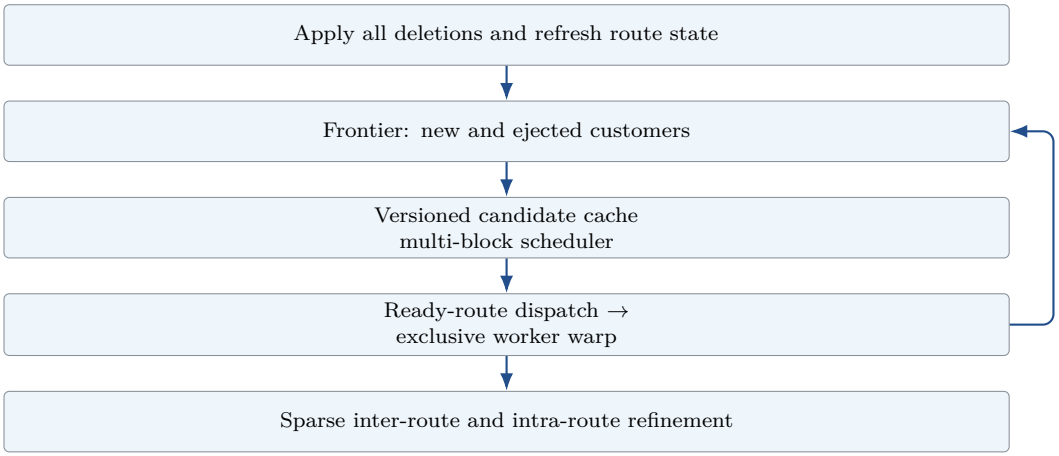


Figure 1: The Runi transaction. An ejection displaces an existing route customer back to the frontier, where it awaits reassignment. Refinement begins only after repair has drained.

4.2 Looking near the customer, not everywhere repeatedly

For route r with L_r customers, Runi maintains a depot-inclusive centroid

$$\mathbf{c}_r = \frac{\mathbf{p}_0 + \sum_{i \in r} \mathbf{p}_i}{L_r + 1}. \tag{3}$$

A centroid gives one constant-size spatial summary per route, so ranking R routes requires $O(R)$ distance checks. Using the nearest *customer vertex* of every route instead would require examining many route members (or maintaining an extra spatial index) each time a new request is assessed. The centroid is therefore

a fast locality heuristic, not a guarantee of the route with the cheapest feasible insertion; a long or irregular route can have a nearby vertex even when its centroid is distant. A newly encountered customer x first compares its location against the currently active route centroids. The $K_c = \min(h_{\max}, 8)$ nearest centroid candidates and their distances are cached. For the experiments $h_{\max} = 5$, hence $K_c = 5$. This cache turns expensive repeated route scans into short lookups when an insertion is retried. A version stamp accompanies each cached candidate: when a route changes, only the affected cached distance must be refreshed.

Two cases need different treatment. If at least one route centroid is no farther from x than the depot, the request enters *local mode*; it considers only the cached candidates, with at most $h_{\max}$ distinct route attempts. This is intentional, not an exact nearest-insertion guarantee: farther routes are omitted to save work. If the depot is closer than every centroid, the request enters *depot mode*, which preserves an exhaustive visited-route fallback before a singleton route is created. The distinction avoids incorrectly abandoning customers simply because the depot is closest to them in the centroid metric. A visited-route mask records real attempts; a momentarily busy route does not consume an attempt.

4.3 The ready-route queue and the worker warps

A selected customer is placed in the bounded priority queue of its chosen route. The route's identifier is then published to a *ready-route queue*, so idle workers can discover exactly which route has work. A worker warp claims one route exclusively, removes its highest-priority pending request, performs the modification, and relinquishes ownership. The ownership protocol prevents two workers from concurrently rewriting the same route; a lightweight reservation also protects the scheduler's queue insertion.

Within a route, 32 CUDA lanes evaluate insertion positions cooperatively. Inserting x between u and v changes distance by

$$\Delta_{\text{ins}}(u, x, v) = d(u, x) + d(x, v) - d(u, v). \tag{4}$$

The worker selects a feasible low-cost insertion using warp-level reduction. If capacity is insufficient, it can instead eject an existing customer y while checking $\sum_{i \in r} q_i - q_y + q_x \le Q$; the displaced request returns to the frontier. When eligible routes have been exhausted, a feasible singleton $(0, x, 0)$ can be created. The repair stage terminates only when no frontier requests, route-queue entries, or active route workers remain. The resulting solution is feasible before refinement begins.

4.4 Multi-block scheduling without losing warp parallelism

For a large route set, a single scheduler warp leaves too much sequential scanning. Runi assigns approximately ρ candidate routes to each scheduler thread; with R possible routes and 32 lanes per warp, the nominal scheduler width is

$$W_s = \left\lceil \frac{R}{32\rho} \right\rceil, \qquad \rho = 8 \text{ routes/thread}. \tag{5}$$

Unlike a one-block scheduler, the route scan is partitioned across cooperating blocks. Block 0 retains coordination and combines per-block candidate summaries, while the other blocks evaluate disjoint route subsets for the *same* frontier customer. This preserves one-customer-at-a-time decisions and limits changes to queue correctness, while extending scanner parallelism beyond the 32-warp limit of one CUDA block.

A second constraint is less obvious. A worker warp temporarily stores a route sequence during mutation. If B warps share a block and the reserved route capacity is S integer entries, its dynamic shared-memory request is $4BS$ bytes. As S grows, keeping the same number of warps *per block* can exceed the available shared memory even though the total number of desired scheduler warps has not changed. Runi therefore varies warp packing independently from W_s. For large route strides, when shared-memory packing would leave only one warp per block, worker scratch is moved to preallocated global memory, allowing the scheduler and worker grid to use wider blocks. The scheduler's cross-block barriers are then shared among fewer blocks, rather than among many single-warp blocks. A residency-aware launch configuration keeps the cooperative scheduling grid resident to prevent barrier deadlock.

5. Sparse GPU Refinement

5.1 Why repair alone is insufficient

The repair phase has a narrow job: make the updated solution feasible quickly. A new customer may be inserted into a locally attractive route without accounting for broader improvements involving neighboring routes. Stopping here would systematically trade away solution quality. Runi therefore adds a distinct refinement stage, called *once per batch*, in which a bounded number of internal passes reshapes the routes after all requests have been placed.

An exhaustive search over all unordered route pairs would examine $R(R-1)/2$ possibilities. Instead, each live route nominates its K_r nearest neighboring *route centroids*; after deduplication, the candidate set contains at most $K_r R$ directed nominations and usually fewer unique undirected pairs. These pairs are

greedily separated into conflict-free matching rounds: no route appears twice in the same round. A warp can then evaluate and commit a pairwise improvement without another warp simultaneously changing either route. Each round is completed before the next begins, so later evaluations see the new route state.

5.2 The three refinement neighborhoods

Each refinement pass uses three ordered stages based on variable-neighborhood-descent (VND) style improvement moves, as implemented in Senay Patel's dynamic-CVRP baseline [2], with the general VND framework described in [7] and SWAP* following the specialized CVRP neighborhood [3]. The numbered move names identify the neighborhoods in the evaluated implementation; they are not universal numbering conventions.

Stage I—Move 8/9, route-tail exchange [2]. Pairs of nearby routes exchange suffixes, considering the relevant orientation of each exchanged tail. This can move entire groups of customers between vehicles and repair a poor division created by sequential insertion. Candidate moves must preserve capacities and decrease total route cost.

Stage II—Move 1 and SWAP* [2, 3]. After Stage I, the centroids and the nearby-pair nominations are rebuilt because route membership has changed. Move 1 relocates a customer between routes. SWAP* exchanges customers between two routes while permitting their best reinsertion positions rather than a fixed position-for-position swap. These moves refine individual customer assignments after the coarser tail exchanges.

Stage III—Move 7, intra-route 2-opt [2]. Each live route is assigned a warp to consider improvements to its internal visiting order. Reversing an appropriate segment can remove an inefficient detour without changing vehicle capacity. Since routes are independent here, many 2-opt evaluations can proceed simultaneously.

A refinement budget P means *up to* P full passes through these stages, with early termination when no improving move is committed. In the results we distinguish **Runi1** ($P = 1$), used to examine low-latency dynamic updates, from **Runi15** ($P = 15$), selected for stronger final-solution quality. The passes are part of the same single refinement *invocation* after a transaction, not one refinement invocation per ADD or DELETE event.

6. Experimental Design

6.1 Benchmarks and measurements

The experiments use CVRPLIB-style instances [5, 6] with depot-to-depot tours, positive customer demands and per-edge rounded Euclidean distances. They test three distinct forms of difficulty: 1,000 customers with relatively few routes (`X-n1001-k43`), 10,000 customers with many short routes (`XL-n10001-k1570`), and 30,000 customers with relatively long routes (`Flanders2`). A larger 100,000-customer instance tests mixed updates, and a 500,000-customer instance tests the ability to accommodate growing event batches. Exact names, capacities and dimensions are collected in the final reference page.

A balanced mixed batch contains m deletions and m additions, resulting in $2m$ events. For the three smaller instances, we report 25 deletions plus 25 additions; for the 100,000-customer case we also report 100 plus 100. The large-scale stress series varies the balanced batch from 50 to 12,800 total events. Reported *algorithm time* refers to the solver's recorded update phases and is distinguished from external wall time, which also includes surrounding application overhead. Only runs for which validity checking succeeded are included as completed scale points.

6.2 Hyperparameters and experimental variants

Table 1 fixes the hyperparameters that otherwise make performance and quality hard to interpret. The same repair configuration is retained when refinement is changed from one to fifteen passes. All scheduler values describe *targets or maxima*; the actual blocks per launch depend on route count and device resources.

Setting	Value	Role
Routes per scheduler thread, ρ	8	Initial centroid-scan target
Local candidate budget, $h_{\max}$	5	Maximum cached route attempts
Route queue capacity	5	Pending requests per route
Refinement neighbors, K_r	5	Nearest route partners
Worker warp target	256	Maximum route workers
Scheduler packing target	8	Warps in a block, adapted to memory
Refinement budget, P	1 / 15	Runi1 / Runi15; early-stop enabled

Table 1: Runi parameters used in the reported comparisons.

6.3 Reference solvers and comparison scope

We consider two baselines that answer different questions. *VND* denotes Senay Patel's Dynamic-CVRP implementation [2], which performs an existing-solution dynamic update followed by a broader CPU variable-neighborhood descent. *seqMDS* [1] is a static, independently optimized solution of the resulting final customer

set. It is useful for observing how an independent restart compares with incremental repair, but its compute budget and timing boundaries are not identical to those of Runi. The algorithms are therefore shown side-by-side rather than described as controlled hardware speedups. For final-state quality, we additionally consult BKS values for the *exact full active instance*, and distinguish them from an HGS rerun; an HGS solution on the *starting* subset is not a reference for the full final set. The HGS method follows Vidal [3, 4].

7. Results

7.1 Scaling with the number of dynamic events

The first question is whether batch processing remains practical as the update itself grows. Figure 2 follows the same 500,000-customer base while the number of deletions and additions increases in equal proportions. The algorithm remains valid through 12,800 total events. Runtime increases approximately linearly over the successful observed points; a fixed refinement component and additional repair retries mean that the curve should not be interpreted as a strict complexity bound.

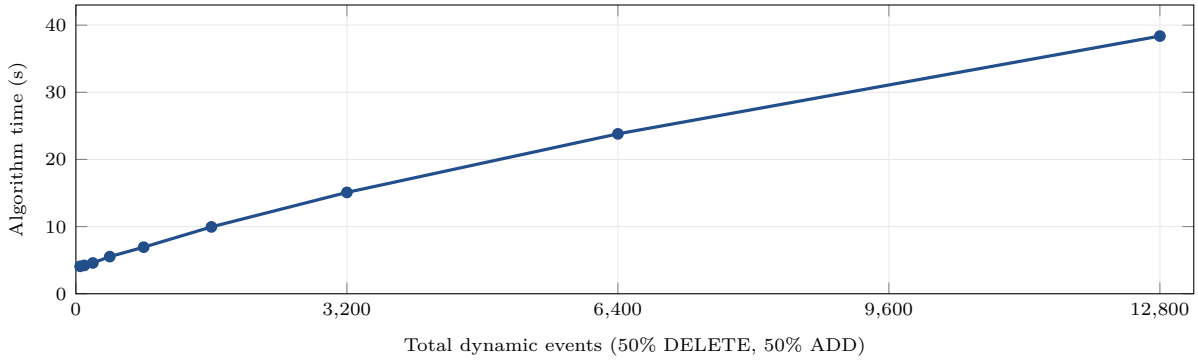


Figure 2: Measured Runi1 batch-update scaling on `XML500000_2366_05`; every point represents a completed valid update.

Events	ADDs	Algorithm (s)	Final cost
50	25	4.085	12,335,987
100	50	4.220	12,348,188
200	100	4.585	12,374,911
400	200	5.529	12,419,019
800	400	6.933	12,486,501
1,600	800	9.954	12,582,454
3,200	1,600	15.091	12,715,321
6,400	3,200	23.811	12,869,226
12,800	6,400	38.370	13,116,377

Table 2: Completed mixed-batch scaling results on `XML500000_2366_05`. All entries passed route validation; failed larger runs are excluded.

The largest completed experiment contains 6,400 cancelled and 6,400 newly added customers, with 38.37 s of algorithm time. Beyond that point, the recorded configuration is bounded by the route-scratch allocation strategy and the shared-memory capability of its RTX 5090 kernel launches. In particular, pairwise refinement needs two temporary route sequences per warp, and the reserved route stride grows conservatively with the number of new customers. The 25,600-event experiments were not completed valid runs and are not plotted as timings. This is a *tested implementation/resource limit*, not a theoretical limit on dynamic CVRP or on all possible GPU algorithms.

One controlled architectural comparison helps explain the scaling result. For the 12,800-event workload, changing a one-warp shared-memory block layout to eight-warp blocks with private global worker scratch reduced cooperating scheduler blocks from 88 to 11, while keeping 88 total scheduler warps. The corresponding ADD time fell from 206.32 s to 31.82 s; both runs passed post-repair and post-refinement route audits. The improvement arises from reducing synchronization across mostly waiting scheduler blocks, rather than from creating more nominal worker warps. These are recorded configurations, not averaged repetitions.

7.2 How many refinement passes are enough?

Increasing P expands the opportunity to improve a feasible repaired solution, but it also adds search time. Table 3 records final costs at four pass budgets on three different route-size regimes. The normalized progress curve in Figure 3 makes the diminishing returns visible without allowing the naturally larger absolute costs of the large instance to dominate the plot.

Instance	Passes	Final cost	Time (s)
`X-n1001-k43`	1	77,166	0.0305
	5	74,708	0.1353
	15	74,122	0.2734
	20	73,972	0.3136
`XL-n10001-k1570`	1	2,362,379	0.0438
	5	2,355,509	0.1453
	15	2,354,029	0.3951
	20	2,353,981	0.5200
`XML100000_2222_01`	1	12,827,656	2.506
	5	12,805,385	11.007
	15	12,799,397	31.779
	20	12,799,148	43.343

Table 3: Recorded refinement-pass sweeps: 25 ADD and 25 DELETE on the first two instances, and 100 ADD and 100 DELETE on the last. Separate from the 80/20 full-instance experiment.

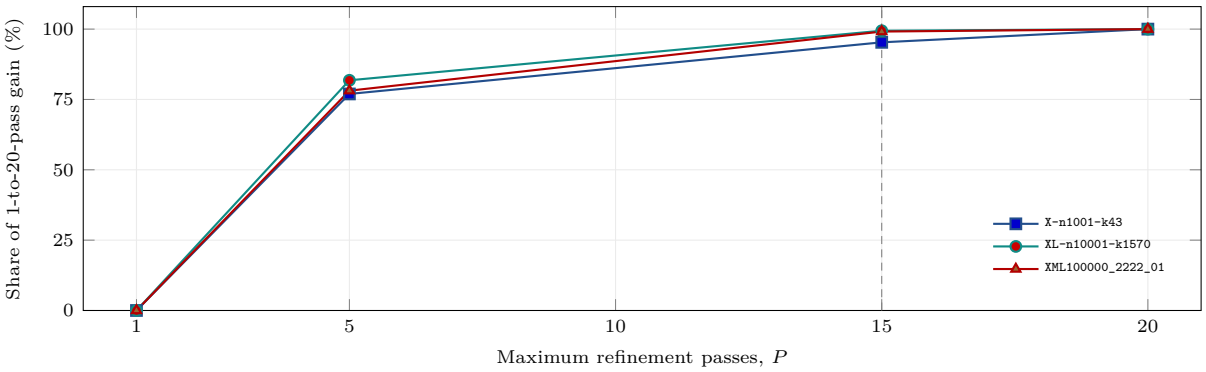


Figure 3: Normalized fraction of the total recorded cost improvement between $P = 1$ and $P = 20$. The vertical guide marks the chosen $P = 15$ setting.

By fifteen passes, the three sweeps have already achieved approximately **95.3%, 99.4%, and 99.1%** of the cost reduction observed by twenty passes. The final five passes reduce the measured 100,000-customer workload's cost by only 249 units while extending its recorded time from 31.78 to 43.34 s. This supports **Runi15** as a practical saturation point for these observations, not an assertion that every workload has an optimum at exactly fifteen passes. Runi1 remains appropriate when update latency is the principal constraint.

7.3 Mixed-update comparison against VND [2] and seqMDS [1]

Table 4 compares identical final-customer workloads under four configurations. Runi1 prioritizes update latency; Runi15 allows more refinement to recover solution quality. On the three 25/25 cases, Runi15 improves Runi1's cost and beats VND [2] on two cases, while remaining close on the third. A fresh seqMDS [1] restart reports higher costs and longer times in this record.

Instance and event batch	Method	Cost	Time (s)
`X-n1001-k43` 25/25	Runi1	77,428	0.02965
	Runi15	**74,122**	0.27340
	VND [2]	75,014	6.82347
	seqMDS [1]	80,277	8.33092
`XL-n10001-k1570` 25/25	Runi1	2,362,379	0.04818
	Runi15	**2,354,029**	0.39507
	VND [2]	2,350,330	14.58895
	seqMDS [1]	2,525,820	119.008
`Flanders2` 25/25	Runi1	4,564,957	2.84205
	Runi15	**4,468,369**	41.18131
	VND [2]	4,502,981	81.04772
	seqMDS [1]	4,910,320	387.659
`XML100000_2222_01` 100/100	Runi1	12,827,656	2.57730
	Runi15	12,799,397	31.779
	VND [2]	—	—
	seqMDS [1]	—	—

Table 4: Recorded mixed-update comparison (25 DELETE + 25 ADD; 100 + 100 for XML). Runi times are GPU update measurements; VND is dynamic CPU and seqMDS is a static restart. Times are not hardware- or search-budget-matched speedups. Dashes indicate that VND and seqMDS did not scale to the XML 100/100 workload in the tested configurations.

Relative to VND, Runi15's final cost is **1.19% lower** on `X-n1001-k43` and **0.77% lower** on `Flanders2`; it is only **0.16% higher** on `XL-n10001-k1570`. Its recorded update times are also shorter in all three cases (0.273 versus 6.823 s; 0.395 versus 14.589 s; 41.181 versus 81.048 s). Thus the fifteen-pass variant substantially narrows, and in two cases reverses, Runi1's quality disadvantage, at higher GPU update cost.

7.4 Quality relative to best-known full-instance solutions

A distinct experiment starts with roughly **80% of a known CVRP customer set**, then adds the remaining **20%**. Since the final active customer set reconstructs the full static CVRP instance, it is meaningful to compare the final result with that instance's BKS, using the same edge-rounding convention. Table 5 reports the recorded Runi15 full-size costs and the stated BKS references. This comparison measures solution-quality gap; it does *not* claim that a published BKS was produced by an online dynamic solver.

Instance	Start / ADD	BKS	Runi15	Gap (%)
`X-n1001-k43`	800 / 200	72,346	76,974	6.397
`XL-n10001-k1570`	8,000 / 2,000	2,333,757	2,426,730	3.984
`Flanders2`	24,000 / 6,000	4,365,762	4,687,091	7.360

Table 5: Runi15 compared with full-instance best-known CVRP costs. Gaps are $(F_{\text{Runi15}} - F_{\text{BKS}})/F_{\text{BKS}} \times 100$. The `Flanders2` BKS has been updated from the 4,373,244 value in an earlier experimental record to the 4,365,762 value reported by CVRPLIB in July 2026 [5].

The 10,000-customer case has the smallest observed gap (3.98%); the 30,000-customer case remains more difficult, consistent with its much longer routes and the fact that refinement considers only spatially nearby route pairs. These gaps should not be interpreted as differences from a *proven* optimum unless the underlying BKS has an optimality certificate.

A separate HGS Advance run [4] produced an *initial* route solution for the 800-customer subset of `X-n1001-k43`: cost 59,403, 34 routes, in 633.41 s. That number is useful for documenting the quality of the starting incumbent, but it cannot be placed into Table 5 as an HGS *final* cost, because the solver had not yet been shown to complete on the reconstructed 1,000-customer set under a matched final-state protocol. The appropriate HGS final column is therefore left unreported rather than inferred from the initial solution.

8. Future Work

References

[1] R. P. Muniasamy, S. Singh, R. Nasre, and N. S. Narayanaswamy, "Effective Parallelization of the Vehicle Routing Problem," *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2023, pp. 1036–1044. doi:10.1145/3583131.3590458.

[2] S. Patel, "Dynamic CVRP Solver & Visualizer," GitHub repository implementing dynamic customer insertion/deletion and variable-neighborhood-descent (VND) reoptimization. `https://github.com/senay73629/Dynamic-CVRP`.

[3] T. Vidal, "Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood," *Computers & Operations Research*, vol. 140, article 105643, 2022. doi:10.1016/j.cor.2021.105643.

[4] T. Vidal, "HGS Advance: standalone hybrid genetic search software," 2026. `https://github.com/tig-foundation/hgs-advance`. The underlying HGS method is documented in [3].

[5] CVRPLIB, "CVRP benchmark instances and best-known solution updates," 2026. `https://galgos.inf.puc-rio.br/cvrplib/en/`; `https://galgos.inf.puc-rio.br/cvrplib/en/updates/`.

[6] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New benchmark instances for the Capacitated Vehicle Routing Problem," *European Journal of Operational Research*, vol. 257, no. 3, pp. 845–858, 2017. doi:10.1016/j.ejor.2016.08.012.

[7] P. Hansen and N. Mladenović, "Variable neighborhood search: Principles and applications," *European Journal of Operational Research*, vol. 130, no. 3, pp. 449–467, 2001. doi:10.1016/S0377-2217(00)00100-4.

Reference Page: Terminology, Parameters, and Instance Sizes

This page collects the principal terms used in the method and the full benchmark identifiers. The table is a reading aid, not an additional experiment.

Glossary

Term	Meaning
Frontier	Device-resident requests waiting for insertion, including displaced customers.
Ready-route queue	Route-ID queue that tells workers exactly which routes have pending work.
Per-route priority queue	Bounded ordered requests for a single route.
Scheduler block	CUDA threads evaluating disjoint route candidates; block 0 also coordinates.
Worker warp	Thirty-two GPU lanes cooperatively and exclusively changing one route.
Route ownership	Atomic exclusion used during route mutation or queue insertion.
Route stride	Reserved slots per route, not actual route length.
Centroid cache/version	Spatial candidate list; version checks detect changes to cached routes.
LOCAL / DEPOT mode	Bounded cached-candidate exploration / exhaustive fallback.
Ejection	Displace a route customer and put that customer back on the frontier.
Matching round	Disjoint route pairs that are safe to refine concurrently.
BKS	Best-known full-instance route cost, not necessarily proven optimal.
Runi1 / Runi15	Same method with at most 1 / 15 refinement passes.

Refinement moves (VND neighborhoods)

The labels below refer to the move implementations used in Runi. The VND baseline is Senay Patelś Dynamic-CVRP code [2]; the general VND framework is in [7], and SWAP* is the customer-exchange neighborhood of [3].

Move	Meaning
Move 8	Exchange suffixes of two routes, reversing the transferred tail orientation considered by this move.
Move 9	Exchange suffixes of two routes, retaining the corresponding tail orientation considered by this move.
Move 1	Relocate one customer from a route into a feasible insertion position of another route.
SWAP*	Exchange one customer from each of two routes, allowing each to be reinserted at its best feasible position rather than at the original positions [3].
Move 7 (2-opt)	Reverse a segment within one route to remove costly detours without changing its customer set.

Experimental hyperparameters (quick reference)

$\rho = 8$ routes/scheduler thread; $h_{\max} = 5$ cached route attempts; **queue capacity** $= 5$; $K_r = 5$ centroid-nearest refinement partners; **worker cap** $= 256$ **warps**; **initial packing** $= 8$ **warps/block**; $P = 1$ **or** 15 refinement passes, with early stop.

Instance register

Benchmark identifier	Customers	Q	Experimental role
`X-n1001-k43`	1,000	131	25/25; 80/20
`XL-n10001-k1570`	10,000	479	25/25; 80/20
`Flanders2`	30,000	200	25/25; 80/20
`XML100000_2222_01`	100,000	37	100/100 pass study
`XML500000_2366_05`	500,000	1,501	Scaling to 12,800 events

Customer count excludes the depot. In the underlying CVRPLIB files, the corresponding `DIMENSION` field includes one additional depot vertex. Values of Q were read from the instance headers.